INFORME TÉCNICO DE AUDITORÍA

Seguridad de aplicaciones web y sistemas de información

Auditoría de Ciberseguridad

GlucoLume

Plataforma experimental de adquisición y análisis de señales ópticas

FastAPI · React · SQL Server · Docker · Arduino

Autora: _____

Licenciatura en Sistemas de Información

Trabajo de titulación

Fecha del informe: 17 de octubre de 2026

Marcos de referencia: OWASP Top 10 (2021) · ISO/IEC 27001:2022 · NIST CSF 2.0

Los resultados de GlucoLume son experimentales y no constituyen medición clínica certificada. Este informe evalúa la seguridad del software y su preparación para despliegue.

Índice

1. Resumen ejecutivo **3**

2. Alcance, materiales y limitaciones **3**

3. Metodología y marcos de referencia **4**

4. Arquitectura y superficies de ataque **4**

5. Estado de los controles de seguridad **5**

6. Hallazgos y matriz de riesgos **7**

7. Mapeo OWASP Top 10 (2021) **10**

8. Recomendaciones priorizadas **12**

9. Resultados de las pruebas ejecutadas por el auditor **13**

10. Conclusión ejecutiva **13**

1. Resumen ejecutivo

Esta auditoría evaluó la seguridad de **GlucoLume**, una plataforma web experimental de adquisición y análisis de señales ópticas (oximetría y estimación de glucosa) compuesta por una API **FastAPI** con **SQLAlchemy/SQL Server**, un frontend **React (Vite)**, un puente serie para dispositivos **Arduino** y despliegue con **Docker Compose**. Los resultados de la plataforma son experimentales y **no se presentan como clínicamente validados**; esta auditoría evalúa la seguridad del software y su preparación para un despliegue, no la validez clínica de las mediciones.

La evaluación combinó: (a) lectura dirigida del código real con evidencia `archivo:línea`; (b) ejecución independiente de la suite de pruebas del proyecto en un entorno aislado; (c) revisión de la configuración de despliegue (Docker, Nginx) y de los informes técnicos previos del propio proyecto, tratados como antecedente y **re-validados contra el código actual**.

Resultado principal: no se encontró ninguna vulnerabilidad de severidad **Alta** confirmada. El sistema demuestra una base de seguridad sólida y coherente: contraseñas con **Argon2id**, autorización centralizada en el backend con aislamiento estricto entre instituciones, validación estricta de entradas (108 usos de `extra="forbid"`), cero usos de SQL dinámico, cero sumideros XSS en el frontend, sesión revocable con rotación de tokens, MFA TOTP cifrado, Google OAuth con state/nonce/PKCE, límites de frecuencia por IP, cabeceras de seguridad, contenedores sin privilegios y una suite de **1398 pruebas automatizadas que aprueban** (37 omitidas por requerir SQL Server real).

Se identificaron **9 hallazgos de severidad Media y Baja** (ninguna Alta), principalmente de configuración y *hardening*, y **6 riesgos residuales** que dependen de la infraestructura real (permisos de la base de datos productiva, copias de seguridad, cifrado en reposo, certificados TLS) y deben cerrarse antes del despliegue. La **preparación para despliegue se califica como CONDICIONADA**: el código está en condiciones razonables; los pasos pendientes corresponden a infraestructura y configuración de producción (sección 8).

---

2. Alcance, materiales y limitaciones

2.1 Alcance examinado

- **Backend** (FastAPI + SQLAlchemy): 12 routers de API con 206 rutas, 48 servicios, 35 modelos, 46 esquemas Pydantic, núcleo de seguridad (`core/security.py`, `core/config.py`), middleware de cabeceras, límite de frecuencia y manejo de errores.
- **Frontend** (React/Vite): cliente HTTP, almacenamiento de tokens, rutas protegidas, guardas de origen, análisis estático de sumideros XSS.
- **Base de datos**: esquema de referencia `GlucoLume_DB_v3.sql` (32 tablas) y scripts incrementales `001`–`029` (solo lectura; **no se tocó la base real**).
- **Despliegue**: `compose.yaml`, `compose.dev/prod/serial.yaml`, plantillas Nginx/Caddy, Dockerfiles.
- **Adquisición**: puente serie Windows (`acquisition/bridge`), protocolo con token compartido, endpoints de telemetría y SSE.
- **Pruebas**: suite completa del proyecto ejecutada por el auditor (76 archivos backend, 45 frontend).

El repositorio contiene dos copias del proyecto (`GlucoLume` y `Glucolume-web`); se auditó `Glucolume-web` (copia con historial Git) y se verificó por hash que los archivos clave son idénticos en ambas.

2.2 Limitaciones declaradas (honestidad de cobertura)

- La auditoría fue **solo lectura**: no se ejecutaron consultas contra SQL Server real, no se inspeccionó el contenido del `.env` productivo, no se reconstruyeron imágenes Docker, no se probaron los certificados ni el dominio reales.
- Los hallazgos se clasifican en **confirmados** (demostrados por código o prueba ejecutada) y **pendientes de validación** (requieren entorno real). Esta distinción se mantiene en todo el informe: ninguna afirmación se presenta como certeza si no hay evidencia detrás.

• La ejecución de pruebas se realizó con Python 3.12 (el proyecto apunta a 3.14) en un entorno virtual aislado fuera del proyecto, con las dependencias del lock `requirements.txt` verificado por hashes.

---

3. Metodología y marcos de referencia

Se aplicó un método por subsistemas con matriz de cobertura: inventario de archivos y superficies de ataque → revisión de cada subsistema con evidencia exacta → contrastación adversarial de los candidatos (el que valida no es el que encontró) → clasificación por riesgo justificado (**probabilidad × impacto**, no por desviación de una lista de verificación).

Los hallazgos se interpretan contra tres marcos reconocidos:

- **OWASP Top 10 (2021)** — para clasificación de vulnerabilidades de aplicación web (sección 7).
- **ISO/IEC 27001:2022** (Anexo A) — para evaluar controles de gestión (sección 5).
- **NIST Cybersecurity Framework 2.0** — para evaluar madurez por funciones: GOBERNAR, IDENTIFICAR, PROTEGER, DETECTAR, RESPONDER, RECUPERAR (sección 5.2).

Herramientas: lectura de código (grep/analítica), Pytest (suite del proyecto), análisis estático de patrones de riesgo en frontend y backend, verificación de configuración Docker/Nginx, y verificación de integridad de copias por hash SHA-256.

---

4. Arquitectura y superficies de ataque

GlucoLume define nueve **límites de confianza** que estructuran la auditoría. La Figura 1 resume el despliegue: el navegador entra por el proxy (Nginx/Caddy), que fija la IP real y aplica cabeceras y límites; el backend, sin puertos publicados, se comunica con SQL Server por ODBC/TLS; el puente serie ingresa con token compartido; OAuth y SMTP salen del backend.

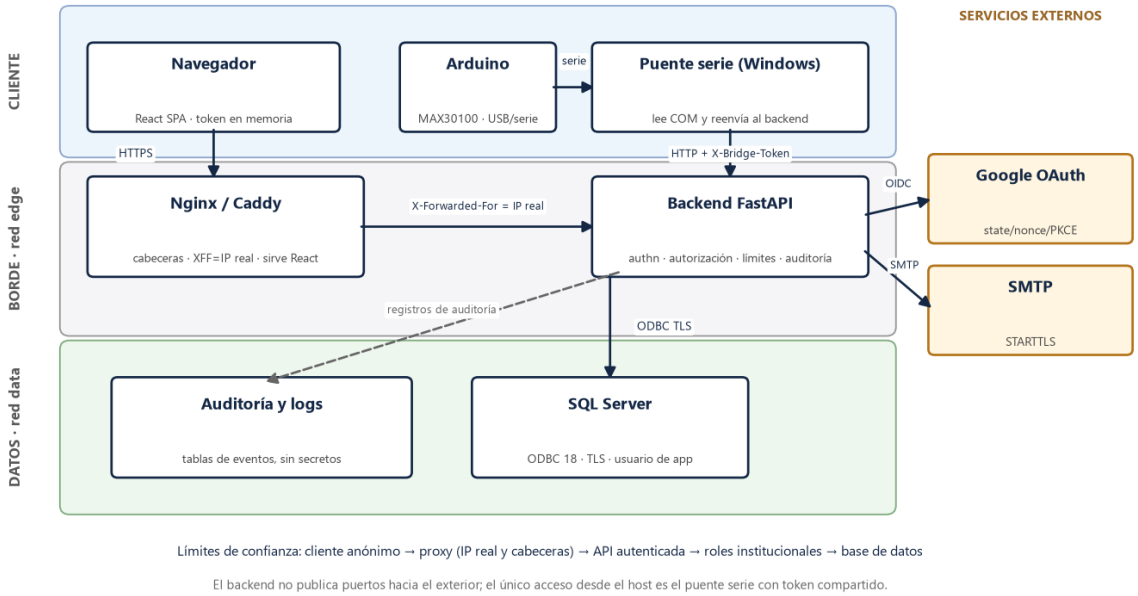


Figura 1. Arquitectura de GlucoLume y límites de confianza (despliegue con Docker).

1. **Cliente anónimo → API pública**: login, registro, recuperación de contraseña, verificación de correo, formulario de contacto, callback de Google. Protegidos por límites de frecuencia por IP (13 categorías en aplicación + zona `auth`

en Nginx con 30 req/s y ráfaga 60).
2. **Cliente autenticado → API privada**: token JWT de acceso (30 min, memoria del navegador) + refresh rotado (7 días) con detección de reutilización; cada request revalida sesión y estado de cuenta contra la base.
3. **Rol dentro de institución**: un único punto de autorización (`requireMembership`, `services/institution.py:241` y `services/group.py:162`) que exige vínculo ACTIVO, institución ACTIVA y permiso del rol (Propietario/Moderador/Operador); a quien no pertenece se le responde «recurso inexistente» (no «prohibido»), evitando filtración de existencia.
4. **Puente serie → API**: endpoints de ingesta protegidos por token compartido de 32+ caracteres validado en configuración.
5. **API → SQL Server**: usuario único de aplicación con pool de conexiones; certificado TLS exigido en producción.
6. **API → SMTP (STARTTLS)** y **API → Google OAuth** (state/nonce/PKCE en cookie firmada, SameSite=Lax, 600 s, Secure en producción).
7. **Nginx/Caddy → backend**: el backend no publica puertos en producción; Nginx fija la IP real del cliente en `X-Forwarded-For` y aplica cabeceras de seguridad y `client_max_body_size 1m`.
8. **Arduino ↔ puente ↔ backend**: protocolo serie propio; autenticidad del dispositivo por códigos de registro y vinculación a instituciones.
9. **Frontend → API**: CORS restringido a un único origen, sin credenciales; autenticación por encabezado Authorization (sin cookies de sesión).

---

5. Estado de los controles de seguridad

5.1 Controles presentes (con evidencia)

Control	Estado	Evidencia
Hash de contraseñas Argon2id con política y recálculo	Presente	core/security.py:20,42,55; prueba ejecutada (suite aprobada)
Autorización centralizada y aislamiento institucional	Presente	services/institution.py:241, services/group.py:162; suite de pruebas de aislamiento aprobada
Validación estricta de entrada (mass assignment bloqueado)	Presente	108 ocurrencias de extra="forbid" en esquemas; pruebas de campos protegidos aprobadas
SQL parametrizado (sin SQL dinámica)	Presente	0 usos de exec_driver_sql ni interpolación de texto en SQL
Mitigación XSS en frontend	Presente	0 usos de dangerouslySetInnerHTML, innerHTML, eval
Límite de frecuencia por IP en endpoints públicos	Presente	api/rateLimit.py:23-37 (13 categorías) + default.conf.template:33,84-86
Cabeceras de seguridad (CSP, nosniff, X-Frame-Options, Referrer-Policy, HSTS condicional)	Presente	api/securityHeaders.py, middleware más externo en main.py:92-93

Control	Estado	Evidencia
Sesiones revocables, rotación de refresh y revocación total ante reutilización	Presente	services/auth.py; pruebas de revocación y cuenta eliminada aprobadas
MFA TOTP con secretos cifrados (Fernet) y códigos de recuperación	Presente	services/mfa.py, core/config.py:51-54
Google OAuth con state/nonce/PKCE	Presente	main.py:35-36,70-77, integrations/google/oauth.py
Ocultación de documentación de API en producción	Presente	main.py:49-56 (404 en API_ENV=production)
CORS de origen único sin credenciales	Presente	main.py:81-90
Secretos fuera del repositorio Git y de las imágenes	Presente	.gitignore cubre .env*; solo se exponen variables VITE_*
Contenedores sin privilegios, redes separadas, límites de recursos	Presente	compose.yaml/compose.prod.yaml (cap_drop: ALL, no-new-privileges, read_only)
Auditoría de eventos con filtrado de secretos y sin endpoints de escritura	Presente	services/audit.py, api/v1/audit.py
Ciclo de vida de cuentas (gracia, eliminación, anonimización, revocación de sesiones)	Presente en código	services/accountLifecycle.py, accountDeletion.py; pruebas SQLite aprobadas

5.2 Mapeo a ISO 27001:2022 (Anexo A, resumen)

Familia ISO 27001	Estado en GlucoLume
A.5 Políticas y gobernanza	Parcial: reglas de negocio y retención documentadas; falta política formal de seguridad aprobada
A.5.15–A.5.18 (proveedores, cumplimiento)	Pendiente: aviso legal de datos de salud, tratamiento de Google como proveedor
A.7 Recursos físicos/humanos	No aplica al alcance (despliegue propio)
A.8.2–A.8.4 Protección de datos, acceso, autenticación	Implementado (hash, MFA, autorización, aislamiento)
A.8.5–A.8.6 Logs y monitoreo, protección del código	Implementado (auditoría de eventos, Git)
A.8.9–A.8.10 Gestión de vulnerabilidades y cambios	Parcial: lock de dependencias con hashes y pip-audit documentado; falta re-escaneo periódico de imágenes
A.8.13 Respaldo de información	Pendiente de verificación (no auditable sin acceso a la infraestructura real)

Familia ISO 27001	Estado en GlucoLume
A.8.20 Seguridad en aplicaciones	Implementado (los controles de la sección 5.1)

5.3 Mapeo a NIST CSF 2.0 (resumen)

Función	Estado
GOBERNAR (GV)	Parcial: reglas documentadas; falta política formal y responsable de riesgo definido
IDENTIFICAR (ID)	Bueno: inventario de activos de datos (32 tablas), roles y flujos documentados
PROTEGER (PR)	Bueno: PR.AA (identidad/acceso), PR.DS (cifrado de secretos y hashes), PR.PS (configuración de contenedores)
DETECTAR (DE)	Bueno en aplicación (auditoría de eventos, avisos de inicio de sesión inusual); falta monitoreo a nivel de infraestructura
RESPONDER (RS)	Parcial: revocación y bloqueo de cuentas automáticos; falta plan de respuesta a incidentes documentado
RECUPERAR (RC)	Pendiente: respaldos y prueba de restauración no verificables sin infraestructura real

6. Hallazgos y matriz de riesgos

Severidad justificada por probabilidad × impacto sobre los activos reales (datos de salud experimentales, cuentas de usuarios, aislamiento institucional). **No se encontró ningún hallazgo Alto confirmado.**

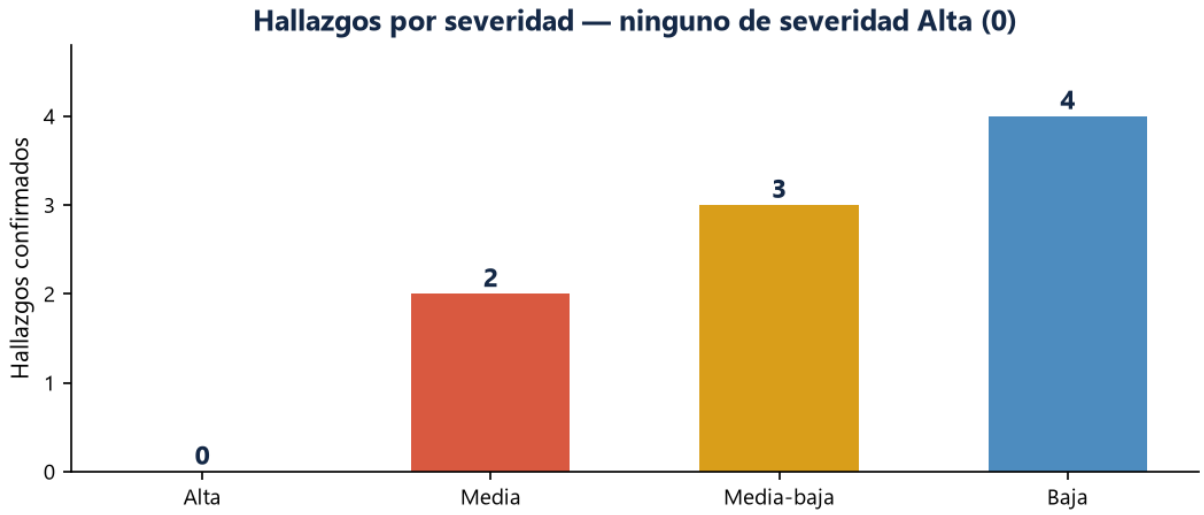


Figura 2. Hallazgos confirmados por severidad (0 Alta, 2 Media, 3 Media-baja, 4 Baja).

ID	Hallazgo	Severidad	Estado	Impacto si se materializa
H-01	API_ENV con valor por defecto development: si un despliegue olvida definirlo, quedan expuestos /docs, /openapi.json, la cookie OAuth sin Secure y la confianza sin verificación del certificado TLS de SQL Server (core/config.py:26,119-122)	Media	Confirmado (configuración)	Exposición del mapa completo de la API y conexión a la base sin verificación de certificado
H-02	Pool de conexiones de SQLAlchemy sin dimensionar (valores por defecto 5+10) frente a hasta 300 streams SSE concurrentes (db/session.py, api/streams.py:22,72)	Media	Pendiente de validación (requiere prueba de carga)	Saturación de conexiones: la API deja de responder bajo ráfagas (disponibilidad)
H-03	FORWARDED_ALLOW_IPS="*" en compose.yaml:48 desalinea la confianza declarada del X-Forwarded-For; hoy no es explotable porque Nginx sobrescribe la cabecera con la IP real, pero sí si el backend se expone directamente o un servicio interno lo llama	Media-baja	Confirmado (hardening)	Evasión de los límites por IP y registro de IP incorrecta en auditoría
H-04	SESSION_SECRET y MFA_ENCRYPTION_KEY sin validación de longitud/formato en configuración (a diferencia de JWT_SECRET y BRIDGE_TOKEN, que sí la tienen) (core/config.py:104-116)	Media-baja	Confirmado (hardening)	Cookie OAuth débilmente firmada; fallo tardío de MFA al descifrar secretos

ID	Hallazgo	Severidad	Estado	Impacto si se materializa
H-05	Parámetros de Argon2id no declarados en el repositorio: se heredan los valores por defecto de la librería y cambian silenciosamente al actualizarla (core/security.py:42)	Media-baja	Confirmado (hardening)	Costo de hashing no versionable; presión de memoria bajo ráfagas de login
H-06	/healthz sin ruta propia en Nginx: hacia afuera respondería el HTML del frontend (docker/nginx/default.conf.template)	Baja	Confirmado (configuración)	Monitoreo externo con señal de salud engañosa
H-07	Contadores de límite de frecuencia en memoria del proceso: con más de una réplica el límite efectivo se multiplica (api/rateLimit.py:10-12)	Baja	Confirmado (deuda técnica, reconocida)	Fuerza bruta distribuida más eficaz en despliegue multi-réplica
H-08	Uso de constante HTTP deprecada (HTTP_422_UNPROCESSABLE_ENTITY) observado como warning al ejecutar la suite (main.py:16)	Baja	Confirmado (deuda técnica)	Ruptura futura al actualizar Starlette
H-09	Menores: detección de deadlock por subcadena de texto de error; CSP sin base-uri/form-action; clasificación de usuario-agente sin recorte de longitud	Baja	Confirmado (hardening)	Falsos positivos en reintentos; endurecimiento CSP incompleto

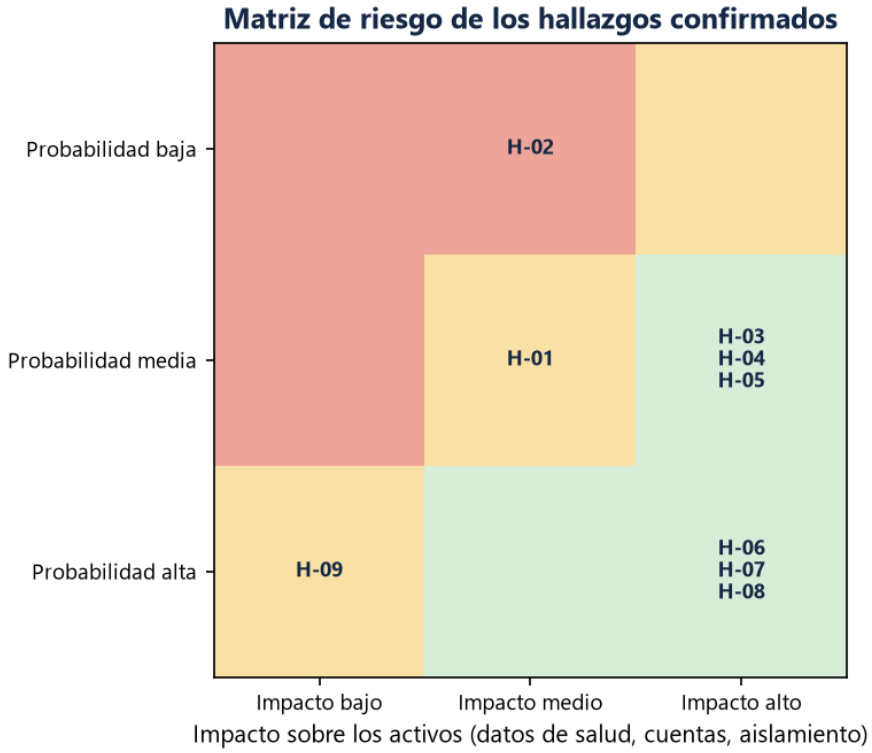


Figura 3. Matriz de riesgo de los hallazgos confirmados (probabilidad × impacto).

6.1 Riesgos residuales (infraestructura real, no cerrables con código)

ID	Riesgo	Estado
R-01	Permisos reales de la cuenta de base de la aplicación: la documentación del proyecto indica que puede UPDATE/DELETE sobre todas las tablas, incluida Auditoria	Pendiente de verificación contra la base real
R-02	Copias de seguridad: existencia, cifrado y prueba de restauración	No verificable sin infraestructura
R-03	Cifrado en reposo (TDE o equivalente) de datos de salud y personales	No verificable sin infraestructura
R-04	Concurrencia del servicio lifecycle sobre SQL Server real (las pruebas correspondientes están entre las 37 omitidas)	Pendiente de validación
R-05	Dominio, certificados Let's Encrypt y URI de redirección de Google reales	Pendiente de verificación
R-06	Re-escaneo de vulnerabilidades de las imágenes Docker finales antes de publicar	Pendiente (requiere reconstruir imágenes)

7. Mapeo OWASP Top 10 (2021)

Categoría OWASP	Evaluación	Fundamento
A01 Control de acceso roto	Cubierto	Autorización centralizada; aislamiento institucional probado; IDOR no encontrado en rutas revisadas
A02 Fallos criptográficos	Cubierto con mejora	Argon2id, Fernet para MFA, SHA-256 para tokens; pendientes: parámetros versionables (H-05) y cifrado en reposo (R-03)
A03 Inyección	Cubierto	100% SQL parametrizado; sin sumideros de interpolación
A04 Diseño inseguro	Cubierto	Reglas de negocio implementadas en servidor (autoría, DEMO vs experimental, cupos, k-mínimo≥5 en estadísticas)
A05 Configuración insegura	Hallazgos H-01, H-03, H-06	Configuración base sólida; los hallazgos son de despliegue
A06 Componentes vulnerables y desactualizados	Cubierto	Locks con hashes; pip-audit/npm audit previos sin hallazgos; pendiente re-escaneo de imágenes (R-06)
A07 Fallos de identificación y autenticación	Cubierto	Bloqueo por cuenta + límites por IP, MFA, rotación de refresh, OAuth con PKCE
A08 Integridad de código y datos	Cubierto	Git con revisión, sin secretos versionados; CI local por suite de pruebas
A09 Fallos de registro y monitoreo	Cubierto en aplicación	Auditoría de eventos con filtrado de secretos y avisos de acceso inusual; falta monitoreo de infraestructura
A10 SSRF	No aplica materialmente	La API no realiza peticiones a URLs proporcionadas por el usuario (solo OAuth fijo y SMTP)

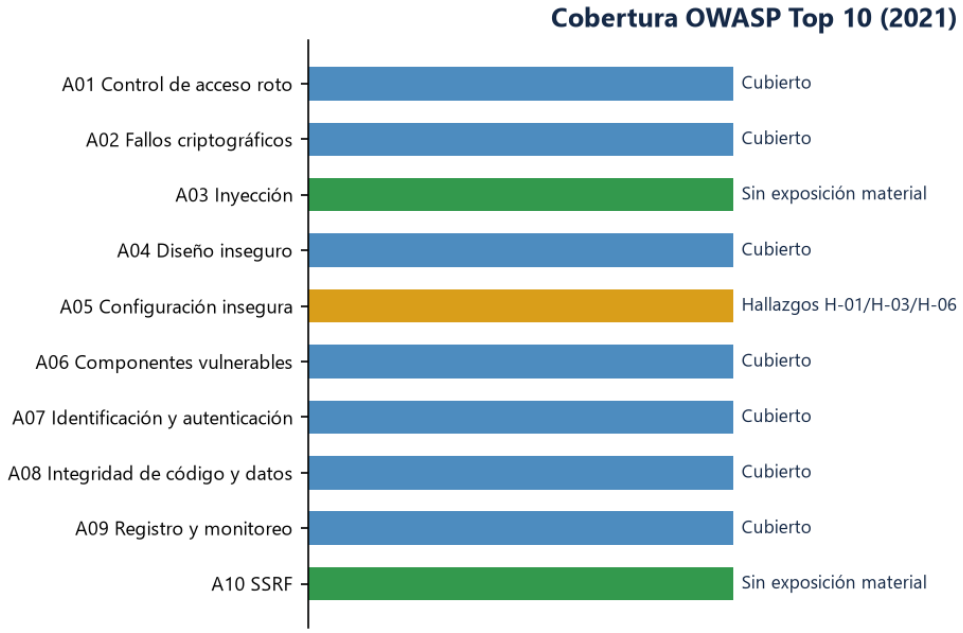


Figura 4. Cobertura OWASP Top 10 (2021) en GlucoLume.

---

8. Recomendaciones priorizadas

P1 — Antes de desplegar

1. Exigir `API_ENV` explícita (cierra H-01). Sin valor por defecto, la aplicación no arranca si falta la variable, eliminando el modo "fail-open":

```
# core/config.py
API_ENV: Literal["development", "test", "production"]   # sin default: error de arranque si falta
```

2. Fijar la confianza del proxy (cierra H-03):

```
# compose.yaml → environment del backend
- FORWARDED_ALLOW_IPS=172.20.0.0/16   # CIDR real de la red "edge" de Docker
```

3. Permisos mínimos en SQL Server y protección de la auditoría (cierra R-01):

```
CREATE USER glucolume_app FOR LOGIN glucolume_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Medicion      TO glucolume_app;
GRANT SELECT, INSERT                 ON dbo.Auditoria     TO glucolume_app;
DENY  UPDATE, DELETE                 ON dbo.Auditoria     TO glucolume_app;
```

4. Respaldo verificado (cierra R-02): respaldo completo semanal + log, cifrado del backup y **prueba de restauración documentada** en un entorno descartable (el proyecto ya tiene la herramienta `database/herramientas/crear_base_descartable.py` para ensayarla).

5. Ejecutar las pruebas de concurrencia sobre SQL Server (cierra R-04): `test_lifecycle_concurrency` y las suites `*_sqlserver` sobre una base de prueba, no productiva.

P2 — Durante la siguiente iteración

6. Validar todos los secretos (cierra H-04):

```
@field_validator("SESSION_SECRET")
@classmethod
def validarSession(cls, value):
```

```
    if value is not None and len(value.get_secret_value()) < 32:
        raise ValueError("SESSION_SECRET debe tener al menos 32 caracteres.")
    return value
```

7. Dimensionar el pool de conexiones (mitiga H-02):

```
create_engine(..., pool_size=10, max_overflow=20, pool_timeout=10, pool_recycle=1800)
```

y medir bajo carga con 100–300 SSE concurrentes.

8. Declarar los parámetros de Argon2 (cierra H-05):

```
Argon2Hasher(time_cost=3, memory_cost=65536, parallelism=4)  # explícito y versionable
```

9. Ruta de salud en Nginx (cierra H-06):

```
location = /healthz { proxy_pass http://backend:8000/healthz; access_log off; }
```

10. Re-escaneo de imágenes antes de publicar (cierra R-06):

```
docker build -t glucolume-backend . && trivy image glucolume-backend
```

P3 — Mejoras continuas

- Redis para los límites de frecuencia si se despliega con más de una réplica (H-07).
- Reemplazar la constante HTTP deprecada (H-08) y agregar `base-uri`/`form-action` a la CSP (H-09).
- Escaneo de Git con una herramienta dedicada (gitleaks/trufflehog) además de los patrones ya usados.
- Plan de respuesta a incidentes de una página (quién responde, cómo se revoca un acceso comprometido, cómo se notifica a usuarios).

---

9. Resultados de las pruebas ejecutadas por el auditor

• **Suite backend completa**: `pytest` → **1398 pruebas aprobadas, 37 omitidas, 0 fallidas** (399 s) en un entorno virtual aislado creado fuera del proyecto, con las dependencias del lock `requirements.txt` instalado en modo `require-hashes`. Las 37 omitidas corresponden a pruebas que requieren SQL Server real (quedan como `needs validation`).

• **Verificaciones estáticas propias**: 0 sumideros XSS en `frontend/src`; 0 usos de SQL dinámica; 108 esquemas con `extra="forbid"`; coincidencia exacta (hash SHA-256) de los archivos de seguridad entre ambas copias del repositorio.

• **Re-validación de los antecedentes**: los tres problemas de severidad Media corregidos en la auditoría interna previa (IP falsificable, falta de límite por IP en endpoints públicos, enumeración en reenvío de verificación) **permanecen corregidos** en el código actual, confirmado por lectura y por las pruebas que los fijan y aprueban.

---

10. Conclusión ejecutiva

GlucoLume presenta, al día de esta auditoría, una **postura de seguridad de aplicación web sólida para un sistema estudiantil-experimental**: la protección de acceso (autenticación, autorización, aislamiento institucional, revocación de sesiones) está implementada en un punto central del backend y respaldada por una suite automatizada extensa que el auditor ejecutó y aprobó de forma independiente. No se encontró ninguna vulnerabilidad de severidad Alta confirmada, ni exposición de secretos, ni inyección SQL, ni sumideros XSS.

Los hallazgos existentes (9, todos de severidad Media o Baja) son mayormente de **configuración de despliegue y hardening**, con correcciones concretas y de bajo costo listadas en la sección 8. Los riesgos que realmente separan al proyecto de un despliegue seguro no están en el código, sino en la infraestructura: permisos finos de la base de datos, respaldos verificados, cifrado en reposo, certificados TLS reales y re-escaneo de imágenes.

Dictamen: preparación para despliegue CONDICIONADA. Completados los cinco pasos P1 de la sección 8, el sistema puede desplegarse como plataforma experimental (manteniendo el aviso de que sus resultados no son clínicamente validados) con un riesgo residual aceptable y trazabilidad de seguridad adecuada para un anexo de tesis.

Informe generado el 17 de octubre de 2026. Evidencia técnica completa (archivos, funciones, líneas y salidas de pruebas) disponible para el tribunal bajo pedido.